# Introduzione

Nel paradigma Divide Et Impera, un algoritmo divide un problema in sottoproblemi, li risolve ricorsivamente e ne combina i risultati. Il tempo di esecuzione di un algoritmo di questo tipo è espresso in termini di un'equazioen di ricorrenza, la cui forma generale è

$$T(n) = aT(n/b) + g(n)$$

Dove:

- $a$ indica in quanti sottoproblemi viene diviso l'input ad ogni passo
- $n/b$ indica di quanto, ad ogni chiamata ricorsiva, si riduce l'input
- $g(n)$ è il costo computazionale speso esclusivamente al livello in esame, per dividere l'input e combinare i risultati, senza considerare le sottochiamate.

È possibile avere una ricorrenza sbilanciata, ad esempio $T(n/2) + T(n/3)$, con fattori di riduzione $b$ diversi, che genererà un albero asimmetrico, in cui alcuni rami sono più corti.

# Il metodo dell'albero di ricorrenza

Per risolvere l'equazione calcolandone l'andamento asintotico, è necessario costruire l'albero seguendo 4 step sequenziali, il costo totale $T(n)$ sarà dato dalla somma dei costi di tutti i nodi.

$$T(n) = a\,T(n/b) + g(n)$$

## Step 1: costruzione dei primi livelli

- Disegnare il nodo radice e assegnargli il costo locale $g(n)$
- Disegnare $a$ nodi figli. L'input per ogni figlio sarà
- Calcolare il costo i ogni figlio sostituendo la nuova dimensione dell'input nella funzione $g$

(SE $g(n) = n^2$, IL COSTO DEL FIGLIO SARÀ $(n/b)^2$)

## Step 2: costo del livello *i* generico

Bisogna trovare il termine generale che esprime quanto costa ogni livello orizzontale

- Numero dei nodi al livello *i*: $a^i$ perché ogni nodo si sdoppia $a$ volte
- Dimensione dell'input al livello *i*: $n/b^i$
- Costo totale del livello: moltiplicare il numero di nodi per il costo locale valutato sul nuovo input. Questo step è fondamentale per impostare la sommatoria finale

## Step 3: calcolare altezza e numero delle foglie

- Altezza (h): porre la dimensione dell'input del livello uguale a 1e risolvi per i, si ottiene $i = \log_b n$
- Numero di foglie: Sostituendo l'altezza nel calcolo dei nodi, l'ultimo livello avrà $a^{\log_b n}$ foglie, che per la proprietà dei logaritmi equivale a $n^{\log_b a}$
- Costo delle foglie: poiché risolvere il caso base costa $\Theta(1)$ il costo dell'ultimo livello è semplicemente $\Theta(n^{\log_b a})$

## Step 4: La somma totale

Il costo totale è la somma di tutti i costi di tutti i livelli interni, più il costo delle foglie. Invece di calcolare la sommatoria esatta, possiamo calcolare il costo dei primi due o tre livelliper capire il fattore di crescita, avremo uno di 3 scenari.

1. I livelli costano sempre di meno ( $n^2 \to \frac{n^2}{3} \to \frac{n^2}{4}$ )
   Il fattore moltiplicativo tra i livelli è strettamente minore di 1. Significa che il costo totale è dominato dalla radice, la complessità è $\Theta(g(n))$
2. Il costo dei livelli aumenta scendendo ( $n \to \frac{3}{2}n \to \frac{9}{4}n$ )
   Il fattore moltiplicativo è maggiore di 1. Significa che dividere il problema costa poco, ma la mole di lavoro vera esplode quando arrivi in fondo a unire un'infinita di casi base. Come risultato, il costo è dominato dalle foglie, la complessità è $\Theta(n^{\text{Foglie}}) = \Theta(n^{\log_b a})$
3. Il costo dei livelli è costante ( $n \to n \to n$ )
   Il fattore moltiplicativo è esattamente 1. Il carico di lavoro è distribuito in modo perfettamente identico su ogni piano dell'albero. Quindi il costo totale è il costo di un singolo livello moltiplicato per l'altezza. La complessità è quindi $\Theta(g(n) \cdot \log_b n)$

# EQUAZIONI DI RICORRENZA SBILANCIATE

PRENDIAMO IN ESAME L'EQUAZIONE

$$T(n) = \begin{cases} \Theta(1) & \text{SE } n \leq 1 \\ a_1 T(n/b_1) + a_2 T(n/b_2) + \dots + g(n) & \text{ALTRIMENTI} \end{cases}$$

LA PRESENZA DI FATTORI DI DIVISIONE DIVERSI GENERA UN ALBERO SINTATTICO SBILANCIATO. IN QUESTI CASI, SPESSO LA NOTAZIONE $\Theta$ NON ESISTE, L'OBBIETTIVO È TROVARE L'INTERVALLO TRA LIMITE SUPERIORE E INFERIORE ($\Omega$ E $O$)

## STEP 1: ESTRAZIONE PARAMETRI

- MOLTIPLICATORI: $a_1, a_2, \dots$
- DIVISORI: $b_1, b_2, \dots$. SE CI SONO FRAZIONI TIPO $\frac{2n}{3}$, RISCRIVERE COME $n/\frac{3}{2}$
- COSTO LOCALE: $g(n)$. SE PRESENTA RADICI, RISCRIVERE COME ESPONENTE FRAZIONARIO ($\sqrt[3]{n^2} \rightarrow n^{\frac{2}{3}}$)

## STEP 2: TEST DEL TREND

CALCOLARE IL COSTO DEI PRIMI 2 LIVELLI DELL'ALBERO PER RICAVARE IL FATTORE MOLTIPLICATIVO DEL COSTO

- COSTO LIVELLO 0: $C_{L_0} = g(n)$
- COSTO LIVELLO 1: $C_{L_1} = a_1 g\left(\frac{n}{b_1}\right) + a_2 g\left(\frac{n}{b_2}\right) + \dots$

CALCOLO DI $q = \dfrac{C_{L_1}}{C_{L_0}}$

## STEP 3: CONFRONTO

- CASO 1: $q < 1 \Rightarrow$ IL COSTO CRESCE SCENDENDO $\Rightarrow \Theta(g(n))$
- CASO 2: $q = 1 \Rightarrow$ TUTTI I LIVELLI COSTANO $g(n)$, QUINDI IL COSTO TOTALE DIPENDE DALL'ALTEZZA $\Rightarrow$ STEP 4
- CASO 3: $q > 1 \Rightarrow$ IL COSTO AUMENTA SCENDENDO $\Rightarrow$ STEP 4

## STEP 4: ANALISI

CALCOLIAMO I DUE SCENARI LIMITE, INDIVIDUIAMO IL $b_i$ PIÙ ALTO (RAMO VELOCE) E IL $b_i$ PIÙ BASSO (RAMO LENTO)

### 1. LIMITE INFERIORE ($\Omega$): RAMO VELOCE ($b_{max}$)

- ALTEZZA MINIMA: $h_{min} = \log_{b_{max}} n$
- SE $q = 1$ (STEP 3.2) $\Rightarrow \Omega(g(n) \cdot \log_{b_{max}} n)$
- SE $q > 1$ (STEP 3.3) $\Rightarrow$ SOSTITUISCO $h_{min}$ NEL TERMINE GENERALE $g(n) \cdot q^h$, OTTENENDO $\Omega(g(n) \cdot q^{\log_{b_{max}} n})$
  [SEMPLIFICARE CON PROPRIETÀ $x^{\log_y z} = z^{\log_y x}$]

### 2. LIMITE SUPERIORE ($O$): RAMO LENTO ($b_{min}$)

- ALTEZZA MASSIMA: $h_{max} = \log_{b_{min}} n$
- SE $q = 1 \Rightarrow O(g(n) \cdot \log_{b_{min}} n)$
- SE $q > 1 \Rightarrow$ SOSTITUISCO $h_{max}$ NEL TERMINE GENERALE, OTTENENDO $O(g(n) \cdot q^{\log_{b_{min}} n}$

## STEP 5 RISULTATO

$$\Omega(\dots) \leq T(n) \leq O(\dots)$$

ESEMPIO

$$T(n) = \begin{cases} \Theta(1) & \text{SE } n < 1 \\ 2T(n/2) + 3T(n/4) + n & \text{ALTRIMENTI} \end{cases}$$

1. $a_1 = 2 \qquad b_1 = 2 \qquad g(n) = n$
   $a_2 = 3 \qquad b_2 = 4$

2.
$$C_{L_0} = n$$
$$C_{L_1} = 2\left(\frac{n}{2}\right) + 3\left(\frac{n}{4}\right) = n + \frac{3}{4}n = \frac{7}{4}n$$
$$q = \frac{\frac{7}{4}n}{n} = \frac{7}{4} \simeq 1.75$$

3. CASO $q > 1$

4.1 LIM INF

- $h_{min} = \log_4 n$

$$\Rightarrow \Omega\left(n \cdot \left(\frac{7}{4}\right)^{\log_4 n}\right) = \Omega\left(n \cdot (n)^{\log_4 \frac{7}{4}}\right)$$
$$= \Omega\left(n^{1+\log_4 \frac{7}{4}}\right)$$

4.2 LIM SUP

- $h_{max} = \log_2 n$

$$\Rightarrow O(n \cdot q^h) = O\left(n \cdot \left(\frac{7}{4}\right)^{\log_2 n}\right) = O\left(n \cdot (n)^{\log_2 \frac{7}{4}}\right) = O\left(n^{1+\log_2 \frac{7}{4}}\right)$$

RISULTATO: $\Omega\left(n^{1+\log_4 \frac{7}{4}}\right) \leq T(n) \leq O\left(n^{1+\log_2 \frac{7}{4}}\right)$

# ESERCIZI IN FORMA $T(n) = a_1 T(\sqrt[b_1]{n}) + a_2 T(\sqrt[b_2]{n}) + g(n)$

NOTA: $\sqrt[b]{n} = n^{\frac{1}{b}}$

1. SOSTITUZIONE

   IMPORRE $n = 2^m$, QUINDI PER DEF. LOGARITMICA $m = \log_2 n$

   QUINDI $\sqrt[b_1]{n} = (2^m)^{\frac{1}{b_1}} = 2^{\frac{m}{b_1}}$ E $\sqrt[b_2]{n} = (2^m)^{\frac{1}{b_2}} = 2^{\frac{m}{b_2}}$

2. DICHIARO UNA NUOVA FUNZIONE $S(m) = T(2^m)$

   $S(m) = a_1 S(m/b_1) + a_2 S(m/b_2) + f(m)$

3. PROCEDERE COL METODO SOPRA

4. ALLA FINE SOSTITUISCO IL RISULTATO CON $m = \log n$